

tests/dynamic/lib/ analyzer_common.sh — operator guide

Revision: 1 **Last modified:** 2026-07-01T00:00:00Z **Status:**
Active sourceable base library for the dynamic data-plane anti-bluff harness (P9). Live-stack consumers honest-SKIP (topology_unsupported) until the dynamic compose profile lands (P10, §11.4.69).

Companion (§11.4.18) to the in-source documentation block at the top of the script. Shared base for every analyzer under tests/dynamic/analyzers/ and every suite under tests/dynamic/suites/.

Overview

A **sourceable** (never executed) base library for the dynamic-mode data-plane analyzers and suites. It does three things:

1. **Sources the committed canonical oracle read-only** — pulls in tests/lib/evidence.sh (assert_egress_ip, assert_graceful_503, assert_cache_hit, assert_no_leak, wg_transfer_delta, ab_pass_with_evidence, ab_skip_with_reason). The analyzers **delegate** to these self-tested helpers — reuse, never reimplement (§11.4.74) — and the library NEVER modifies the committed lib (§11.4.58 file-lane).
2. **Provides verdict-emit + self-test TAP helpers** so each analyzer can prove it PASSES its golden-good fixture AND FAILs its golden-bad fixture (§11.4.107(10) self-validated analyzer; an analyzer that passes its own negation is a bluff gate).
3. **Provides the live-stack availability probe + honest SKIP** the P10 suites use, so an absent dynamic stack is a §11.4.69 SKIP-with-reason, never a fake PASS.

Prerequisites

- POSIX sh, awk, grep, tr on PATH.
- curl — **only** for the live-stack reachability probe (the P10 path).

- The committed tests/lib/evidence.sh present at the repo root. When it is absent, AC_EVIDENCE_AVAILABLE=0 and delegating analyzers FAIL honestly rather than bluff a PASS.

Usage examples

```
# Source it from an analyzer or suite (do NOT exec it):
. tests/dynamic/lib/analyzer_common.sh

# Typical consumer patterns:
ac_pass "my_signal" "[evidence: $artefact]"
    # emit a PASS line, rc 0
ac_fail "my_signal" "[reason: <why>]"
    # emit a FAIL line, rc 1
dyn_skip_if_no_stack "stress" && return 0
    # honest SKIP when no stack
dyn_run_analyzer graceful_503_analyzer.sh "$manifest" # run an
    analyzer as subprocess
dyn_red_mode && echo "RED baseline" # §11.4.115
    polarity test
```

Environment knobs read by the library:

- HELIX_DYNAMIC_STACK — 1 declares the live dynamic stack is up (P10).
- HELIX_PROXY_URL — proxy URL the suites drive (default http://127.0.0.1:34128).
- HELIX_PROBE_TIMEOUT — reachability-probe timeout in seconds (default 5).
- RED_MODE — 1 selects the §11.4.115 RED-baseline polarity.

Edge cases

- **Sourced, not exec'd** — \$0 is the caller, so the library resolves its own directory by a fallback search (lib/, ../lib/, ../../tests/dynamic/lib/); a path it cannot resolve degrades gracefully.
- **evidence.sh missing** → AC_EVIDENCE_AVAILABLE=0; consumers that delegate report committed tests/lib/evidence.sh not found — cannot delegate and FAIL, never silently pass.
- **Stack declared but proxy unreachable** → dyn_live_stack_available returns non-zero and dyn_skip_if_no_stack emits a network_unreachable_external SKIP (distinct from the topology_unsupported SKIP used when the stack is simply not declared).
- **Counters** use VAR=\$((VAR+1)) (always ≥1) so the increment never trips the set -e arithmetic-zero abort.

Internal behaviour

- `#!/usr/bin/env bash` shebang but **POSIX-clean** — parses under both `sh -n` and `bash -n` (§11.4.67); no bash-only constructs (`[[ ]]`, `<<<`, arrays, `>( )`, `${v^^}`).
- **Verdict emitters:** `ac_pass` (rc 0), `ac_fail` (rc 1) mirror the `evidence.sh` structured-line contract (PASS: / FAIL: / SKIP:).
- **Self-test TAP harness:** `ac_selftest_reset`, `ac_expect <rc> <desc> -- cmd...` (captures rc, prints TAP ok/not ok, surfaces the verdict on failure), `ac_selftest_summary <name>` (emits the 1..N plan + RESULT line; rc 0 iff every assertion passed).
- **Live-stack helpers:** `dyn_stack_proxy_url`, `dyn_live_stack_available` (issues one short `curl -x` reachability probe only when `HELIX_DYNAMIC_STACK=1`), `dyn_skip_if_no_stack <desc>`.
- **Plumbing:** `ac_qa_dir <sub>` (gitignored qa-results/ evidence dir), `dyn_analyzers_dir`, `dyn_run_analyzer <basename> <args...>` (invokes an analyzer as a fully decoupled subprocess), `dyn_red_mode`.

Related

- `tests/lib/evidence.sh` — the committed canonical data-plane oracle this library sources read-only.
- The six analyzers under `tests/dynamic/analyzers/*.sh` and the six suites under `tests/dynamic/suites/*.sh` — every one sources this base.
- `tests/dynamic/analyzers/run_analyzer_selftests.sh` — drives the self-test harness across all analyzers.
- Design spec §13/§14: `docs/superpowers/specs/2026-06-30-vpn-aware-proxy-extension-design.md`.
- Constitution §11.4 / §11.4.69 / §11.4.107 / §11.4.115 / §1.1.

Last verified

2026-07-01 — `sh -n + bash -n parse-clean`. Exercised today (no live stack, no network) via the bundled fixtures through `run_analyzer_selftests.sh`; the live-stack probe path is exercised in **P10**.